



Random Maze Solver

Trovare l'uscita di un labirinto con il movimento casuale di "particelle"

Lorenzo Cappellini

Parallel Computing

Il problema: Random Maze Solver

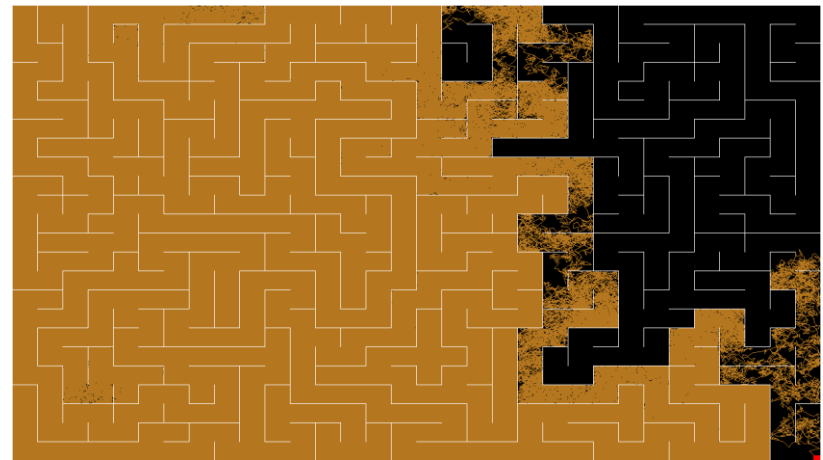
- Obiettivo: **trovare l'uscita** di un labirinto simulando **particelle** che si muovono **casualmente**
 - Partenza → movimento casuale → rimbalzo sui muri → uscita
- Si lanciano **molte particelle** contemporaneamente
 - La prima che esce **interrompe** la ricerca → si ricostruisce il suo percorso (backtracking)
- Studio sulle prestazioni con **parallelizzazione multithread** → **OpenMP**

Esempio di labirinto

Esito diverso a seconda della fortuna del cammino casuale



Caso ottimo (350000 steps)



Caso pessimo (5 milioni di steps)

Flusso del programma

- Genera il labirinto di partenza
- Avvia **n threads** → ciascuno lancia **p particelle** indipendenti
- Ogni particella avanza con **step casuali**
- Prima particella che trova l'uscita → misura **tempo e iterazioni**
- **Ripete la misura** più volte per ogni coppia (p, n)
- Salva tutte le misure in un file .csv

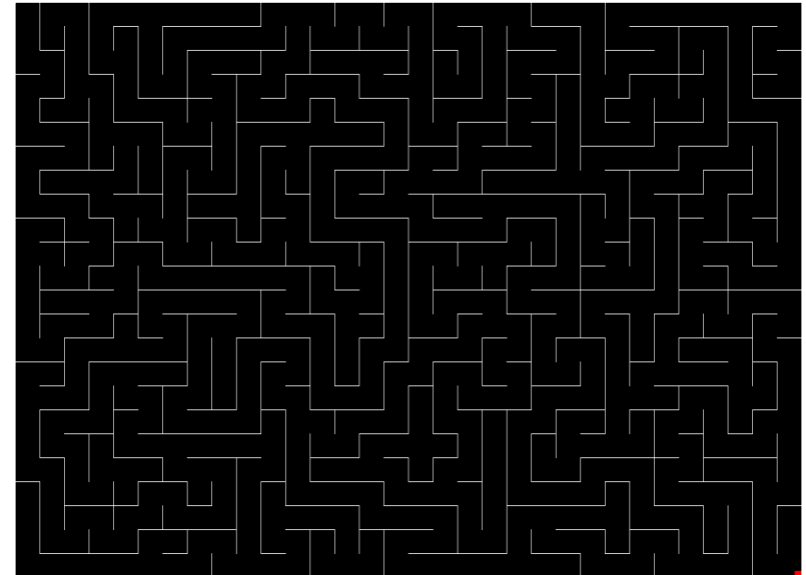


Implementazione: classi principali

- **Point**
 - Usata per salvare le **coordinate (x, y)** della particella
- **Rect**
 - Implementa un rettangolo geometrico, utile per identificare la **regione di arrivo** (uscita del labirinto); può verificare se un **Point** è al suo **interno**
- **Path**
 - Rappresenta il **percorso di una particella** (vector di Point)
- **StepGenerator**
 - Implementa un **generatore casuale** (RNG) con distribuzione uniforme per gli **step**
- **Maze**
 - **Genera** il labirinto, **valida i movimenti** delle particelle considerando i **rimbalzi sui muri**; contiene ulteriori metodi per disegnare il labirinto con *OpenCV*
- **MazeSolver**
 - Implementa l'algoritmo di **creazione, gestione e avanzamento delle particelle**
 - Distribuisce il lavoro sui **thread**
 - **Misura** i tempi impiegati

Generazione del labirinto

- **Algoritmo: Recursive Backtracking (sequenziale)**
- Partendo dalla cella (0,0)
 - si visitano ricorsivamente le celle adiacenti non visitate, in ordine casuale
 - si rimuove il muro tra cella corrente e cella appena raggiunta
- Un **unico percorso** possibile dall'ingresso all'uscita
- Il labirinto viene **generato una sola volta** e salvato su file
 - **stesso labirinto** ad ogni run → misure più confrontabili



Esempio di labirinto generato con *Recursive Backtracking*

Parallelizzazione OpenMP

- Ogni **particella è indipendente** dalle altre
 - particelle indipendenti → **parallelizzabili** facilmente con **OpenMP**
- Ogni thread avanza di uno step per volta un certo numero di particelle
 - Labirinto in **sola lettura** → **nessuna sincronizzazione** necessaria **sui dati**
 - Ogni particella scrive solo sul proprio percorso → **nessuna race condition** in scrittura
- Unica **sincronizzazione** richiesta: la **prima particella** che arriva all'**uscita** deve **fermare tutte le altre**

Sincronizzazione tra i thread

`#pragma omp parallel for num_threads(n)`

- Annota il ciclo esterno sugli “agenti”: un **thread per agente**
- Unica **variabile condivisa** in scrittura: ***running***
 - implementata come **variabile booleana atomica** (`std::atomic`)
 - particella arriva all'uscita → **imposta a false**
 - tutti gli altri thread **controllano running ad ogni iterazione** → si fermano se falsa

```
1 #pragma omp parallel for num_threads(n)
2 for (int t = 0; t < n; t++) { // 1 ciclo per ogni thread
3     // inizializzazione vettori di generatori, paths e iterazioni
4     while (running.load(std::memory_order_relaxed)) {
5         for (int i = 0; i < (int)p; i++) { // per ogni particella del thread
6             // genera prossimo punto (p2) e calcola i rimbalzi
7             // incrementa iterazioni
8             if (end.inside(p2)) { // entrata nella regione di arrivo
9                 bool expected = true;
10                if (running.compare_exchange_strong(expected, false, std::memory_order_relaxed)) {
11                    // salva iterazioni e percorso trovato
12                }
13                break;
14            }
15        }
16    }
17 }
18 // calcola tempo trascorso
19 // ritorna iterazioni e tempo
```

Snippet semplificato del codice di avanzamento delle particelle

Valutazione prestazioni: Difficoltà riscontrate

- Il moto delle particelle è **casuale**
 - stessa configurazione $(p, n) \rightarrow$ **risultati incoerenti** tra loro
 - **Tentativo 1:** fissare un **seed** per gli *RNG* (*StepGenerator*) $\rightarrow$ elimina la componente randomica $\rightarrow$ **snatura il problema** $\rightarrow$ scartato
 - **Tentativo 2:** **ripetere le misurazioni** più volte
 - **Tentativo 3:** utilizzo della **mediana** $\rightarrow$ più **robusta** della media **rispetto agli outlier**

Valutazione prestazioni

Il carico di lavoro non è una sola variabile, ma due:

- **Numero di particelle per thread (p)**
 - **p aumenta** → più tentativi indipendenti → **più probabilità di un percorso migliore** → **tempo diminuisce**
- **Numero di thread (n)**
 - **n aumenta** → più parallelismo → più agenti (e particelle) cercano l'uscita contemporaneamente → **tempo diminuisce**
- **Totale particelle lanciate $P_{tot} = p \times n$**
- CPU utilizzata: *AMD Ryzen 7 3700X* – **8 core fisici, 16 thread logici (SMT)**
- Valori: **$p \in \{1...20\}$, $n \in \{1...16\}$ → 320 configurazioni totali**
- **100 ripetizioni** per ciascuna configurazione

Risultati: numero di iterazioni

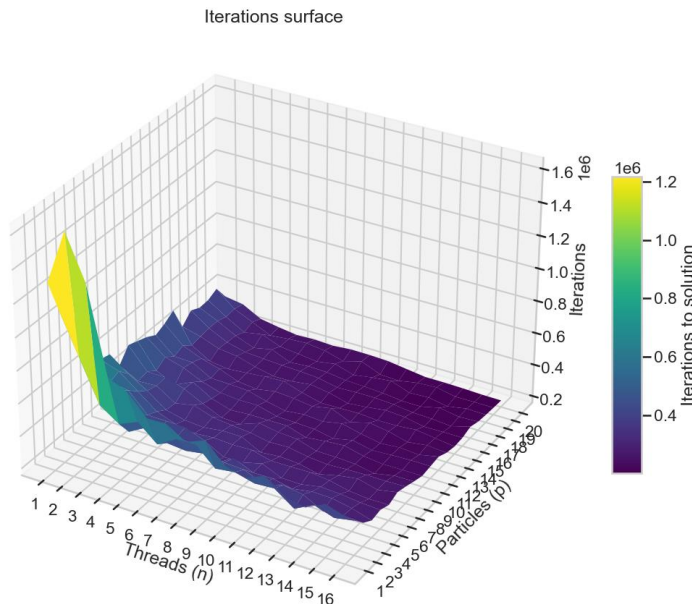


Grafico del numero di step della particella vincente al variare di p e n

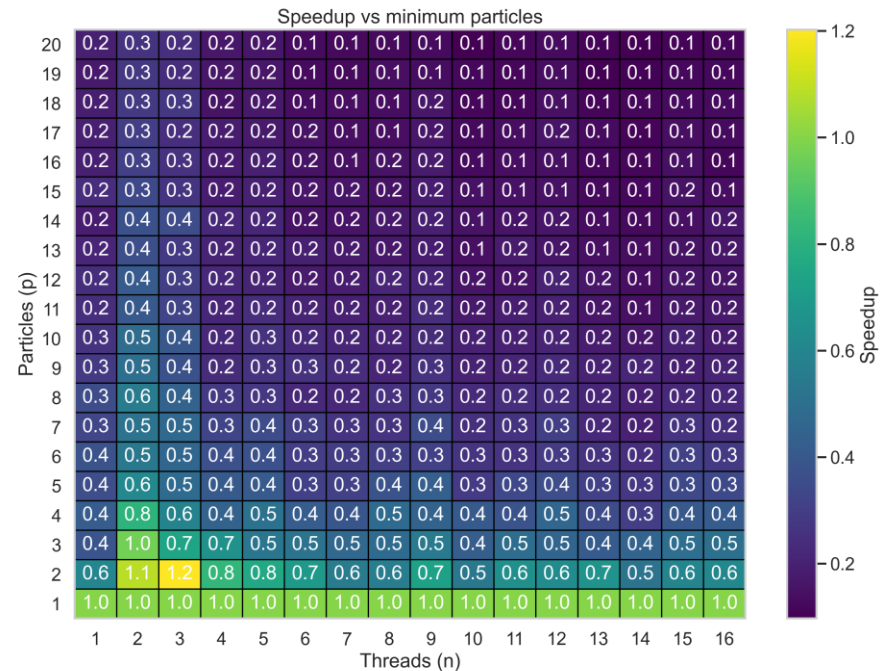
- **Più particelle** → più probabilità di trovare una **soluzione con minor numero di step**
- **Più particelle** → iterazioni totali aumentano (**più carico di lavoro totale**)
- Decrescita proporzionale a $1/P_{tot}$ con $P_{tot} = p \times n$

Risultati: speedup vs singola particella

- Formula:

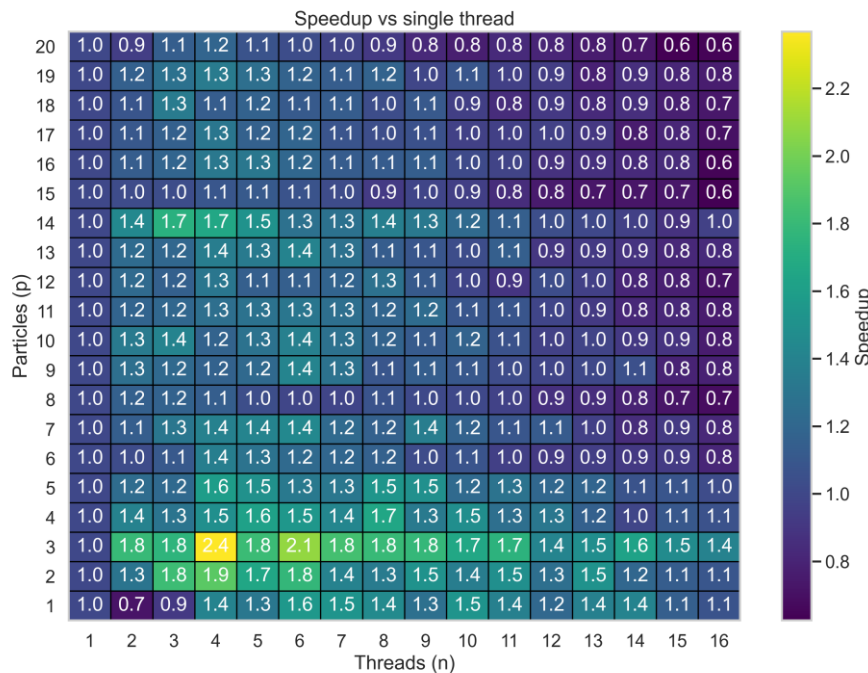
$$S_1(p, n) = \frac{T(1, n)}{T(p, n)}$$

- Misura l'effetto dell'**aumento di particelle**
- p aumenta** → **carico** di lavoro del thread **cresce** linearmente
- Più particelle aiutano a trovare un percorso migliore, ma il costo della computazione prevale → **speedup decresce**



Heatmap del valore di speedup (S_1) al variare di p e n

Risultati: speedup vs singolo thread



Heatmap del valore di speedup (S_2) al variare di p e n

- Formula:

$$S_2(p, n) = \frac{T(p, 1)}{T(p, n)}$$

- Misura l'effetto dell'aumento di thread
- Più thread → più ricerche contemporanee → più probabilità di trovare l'uscita velocemente
- Picco tra $n=4$ e $n=8$
 - oltre $n=8$ → SMT: thread condividono cache → contesa → speedup scende

Risultati: speedup complessivo

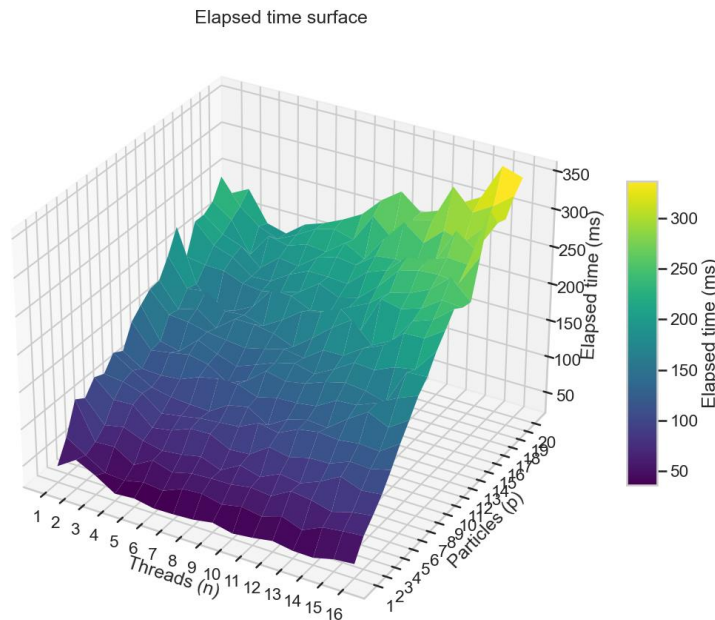
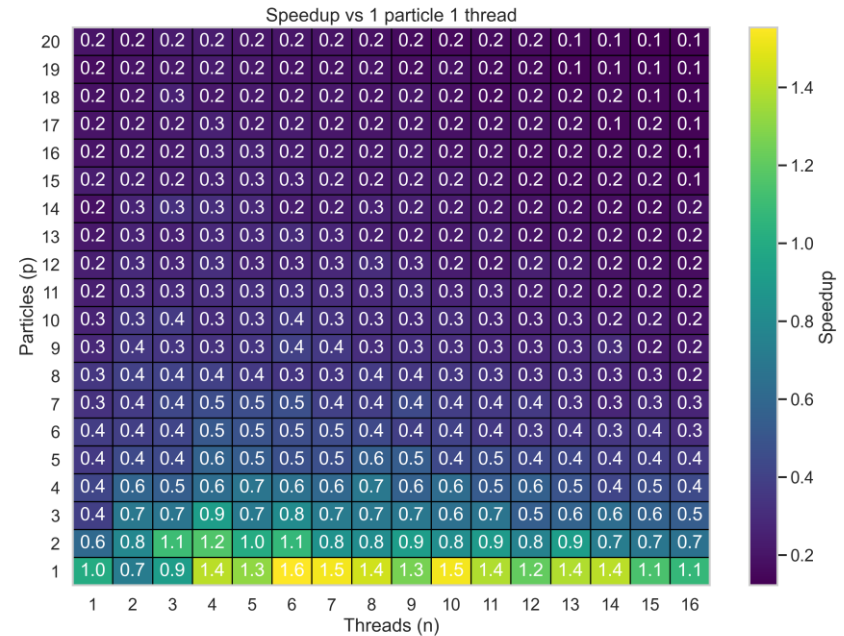


Grafico del tempo impiegato al variare di p e n



Heatmap del valore di speedup (S_3) al variare di p e n

- Formula:
$$S_3(p, n) = \frac{T(1, 1)}{T(p, n)}$$

- Il tempo assoluto cresce molto con p , mentre con n rimane pressoché costante fino a $n=8 \rightarrow$ picchi speedup con $p=2$

Conclusioni

- L'**elevata variabilità intrinseca** del problema rende **difficile** l'**analisi** delle prestazioni
- **Più particelle** → **percorso** finale molto **più corto** ma con **costo computazionale** maggiore
- **Più thread** → speedup reale, ma si degrada oltre i core fisici della macchina
- → nonostante ciò il problema affrontato si presta comunque al **parallelismo**, che con il **giusto compromesso** porta benefici
- **Configurazione ottimale:**
 - **$p = 2$ particelle per thread, $n = 8$ thread (pari ai core fisici)**



Random Maze Solver

Trovare l'uscita di un labirinto con il movimento casuale di "particelle"

Lorenzo Cappellini

Parallel Computing